SCANNER

Compile Design

Prepared By

Student Name	Student ID
Omar Moustafa Megahed	**200023301**

Under Supervision

Dr: Nehal Abd Elsalam

T. A.: Mariam Freed

Summary

1. Overview

This documentation describes the modifications made to adapt the original TINY language scanner to support C- lexical elements, along with instructions to run the scanner and generate tokenized output. The scanner now recognizes C- syntax, including modern operators, control structures, and comment styles.

2. Modifications Summary

Key Changes

1. **Lexer Rules (TINY.flex):**
   - **Removed TINY-Specific Elements:**
     - **Keywords: then, end, repeat, until, read, write.**
     - **Operators: := (replaced with = for assignment).**
   - **Added C- Lexical Elements:**
     - **Keywords: int, void, return, while, do, for, break, continue.**
     - **Operators: ==, !=, &&, ||, !, +=, -=, *=, /=, ++, --.**
     - **Brackets: [] for arrays.**
     - **Comments: // (single-line) and /* */ (multi-line).**
2. **Token Definitions (tokens.h):**
   - **Updated TokenType enum to include C- keywords and operators (e.g., BREAK, CONTINUE, PLUSEQ).**
3. **Utility Functions (util.c):**
   - **Updated printToken to handle new operators and keywords.**
   - **Removed TINY-specific syntax tree nodes (e.g., RepeatK, ReadK).**

3. Running the Scanner

Prerequisites

- **Flex (Fast Lexical Analyzer) installed.**
- **C compiler (e.g., gcc).**

Steps

1. **Navigate to Project Directory:**

```
cd /path/
```

2. **Compile the Scanner:**

```
flex TINY.flex        # Generate lex.yy.c from TINY.flex
```

3. **Run the Scanner:**

```
.\scanner_new.exe
```

The scanner will read input from tiny.txt and print tokenized output to the console.

---

4. Example Input & Output

Input File: tiny.txt

```
/* Test C- specific lexical elements */
int y;
y = 18;
y++;
y--;
y += 10;
y -= 5;
y *= 2;
y /= 4;

if (y > 0 && y < 30)
```

```
  y = y + 1;

if (y <= 20 || y >= 40)
  y = y - 1;

if (!y)
  y = 1;

int arr[5];
arr[0] = 1;
arr[4] = 5;

while (y > 0)
  y = y - 1;

/* Test comment styles */
// Single line comment
/* Multi-line
   comment test */

void func(int param)
  return;
```

Generated Output

```
2: reserved word: int
        2: ID, name= y
        2: ;
```

3: ID, name= y

3: =

3: NUM, val= 18

3: ;

4: ID, name= y

4: ++

4: ;

5: ID, name= y

5: --

5: ;

6: ID, name= y

6: +=

6: NUM, val= 10

6: ;

7: ID, name= y

7: -=

7: NUM, val= 5

7: ;

8: ID, name= y

8: *=

8: NUM, val= 2

8: ;

9: ID, name= y

9: /=

9: NUM, val= 4

9: ;

11: reserved word: if

11: (

11: ID, name= y

11: >

11: NUM, val= 0

11: &&

11: ID, name= y

11: <

11: NUM, val= 30

11:)

12: ID, name= y

12: =

12: ID, name= y

12: +

12: NUM, val= 1

12: ;

14: reserved word: if

14: (

14: ID, name= y

14: <=

14: NUM, val= 20

14: ||

14: ID, name= y

14: >=

14: NUM, val= 40

14:)

15: ID, name= y

15: =

15: ID, name= y

15: -

15: NUM, val= 1

15: ;

17: reserved word: if

17: (

17: !

17: ID, name= y

17:)

18: ID, name= y

18: =

18: NUM, val= 1

18: ;

20: reserved word: int

20: ID, name= arr

20: [

20: NUM, val= 5

20:]

20: ;

21: ID, name= arr

21: [

21: NUM, val= 0

21:]

21: =

21: NUM, val= 1

21: ;

22: ID, name= arr

22: [

22: NUM, val= 4

22:]

22: =

22: NUM, val= 5

22: ;

24: reserved word: while

24: (

24: ID, name= y

24: >

24: NUM, val= 0

24:)

25: ID, name= y

25: =

25: ID, name= y

25: -

25: NUM, val= 1

25: ;

32: reserved word: void

32: ID, name= func

32: (

32: reserved word: int

32: ID, name= param

32:)

33: reserved word: return

33: ;

33: EOF

Technical Report: C- Lexical Scanner Implementation

1. Introduction

This report documents the development of a lexical analyzer (scanner) for the C- programming language, a subset of C designed for educational use. The scanner is a critical component of a compiler, responsible for tokenizing source code and validating syntax at the character level.

1.1 Phases of a Compiler

A compiler operates in six primary phases (Figure 1):

1. **Lexical Analysis: Converts characters into tokens.**
2. **Syntax Analysis: Validates structure using a grammar.**
3. **Semantic Analysis: Checks for logical consistency.**
4. **Intermediate Code Generation: Produces abstract representations.**
5. **Optimization: Enhances code efficiency.**
6. **Code Generation: Outputs target machine code.**

This project focuses on Phase 1 (Lexical Analysis).

2. Lexical Analyzer

2.1 Role in Compilation

The lexical analyzer:

- **Reads source code character-by-character.**
- **Groups characters into valid tokens (e.g., keywords, identifiers).**
- **Removes whitespace/comments.**
- **Reports lexical errors (e.g., invalid symbols).**

2.2 Modifications for C-

The original TINY language scanner was adapted to support C- syntax. Key changes include:

Table 1: Lexical Element Comparison

Category	TINY	C- (Updated)
Assignment	:=	=
Equality	=	==
Logical Ops	None	&&, `, !`
Compound Ops	None	+=, -=, ++, --
Keywords	repeat, read	int, void, while, for

Key Enhancements

1. **Comment Handling: Supports // (single-line) and /* */ (multi-line).**
2. **Error Detection: Flags invalid tokens (e.g., standalone :).**
3. **Line Tracking: Reports line numbers for debugging.**

3. Software Tools

3.1 Tools Used

- **Flex: Generates the lexical analyzer from rules defined in TINY.flex.**
- **GCC: Compiles the generated C code into an executable (scanner_new.exe).**

Table 2: Toolchain Workflow

Step	Command	Output
1	flex TINY.flex	lex.yy.c
2	gcc lex.yy.c -o scanner_new.exe -lfl	Executable scanner

3.2 Running the Scanner

1. **Navigate to the project directory:**

bash

Copy

Download

cd /path/to/scanner

2. **Execute:**

bash

Copy

Download

.\scanner_new.exe

3. **The scanner reads tiny.txt and outputs tokens (Figure 2).**

4. Conclusion

The adapted scanner successfully tokenizes C- code, handling modern syntax elements like compound operators and comments. It serves as a robust foundation for later compiler phases. Future work includes integrating the scanner with a parser for syntactic validation.